A
Project Report
On
"VIRTUAL POINTER"

Prepared by

SMIT PATEL (19DCS103)
YASH PATEL (19DCS106)

Under the guidance of

Rima Patel

Assistant Professor

A Report Submitted to

Charotar University of Science and Technology

for Partial Fulfillment of the Requirements for the

3rd Semester Software Group Project-I (CE244)

Submitted at

CSE

DEPSTAR

At: Changa, Dist: Anand – 388421

JUNE-DECEMBER 2020

CERTIFICATE

This is to certify that the report entitled "**VIRTUAL POINTER**" is a bonafied work carried out by **SMIT PATEL (19DCS103) & YASH PATEL (19DCS106)** under the guidance and supervision of **Assistant Prof. Rima Patel** for the subject CE244 - **Software Group Project-I** (CSE) of 3rd Semester of Bachelor of Technology in **DEPSTAR** at Faculty of Technology & Engineering – CHARUSAT, Gujarat.

To the best of my knowledge and belief, this work embodies the work of candidate himself, has duly been completed, and fulfills the requirement of the ordinance relating to the B.Tech. Degree of the University and is up to the standard in respect of content, presentation and language for being referred to the examiner.

Rima Patel
Assistant Professor
CSE
DEPSTAR, Changa, Gujarat.

Dr. Amit Ganatra
Principal, DEPSTAR
Dean, FTE
CHARUSAT, Changa, Gujarat.

Devang Patel Institute of Advance Technology and Research At: Changa, Ta. Petlad, Dist. Anand, PIN: 388 421. Gujarat

DECLARATION BY THE CANDIDATE

I hereby declare that the project report entitled “Virtual Pointer” submitted by me to Devang Patel Institute of Advanced Technology an Research, Changa in partial fulfilment of the requirement for the award of the degree of B.Tech in Computer Science and Engineering, from Department of Computer Science and Engineering, DEPSTAR/FTE, is a record of bonafied CE244 Software Group Project-I (project work) carried out by me under the guidance of Prof. Rima Patel. I further declare that the work carried out and documented in this project report has not been submitted anywhere else either in part or in full and it is the original work, for the award of any other degree or diploma in this institute or any other institute or university.

Smit Patel (19DCS103)
Yash Patel (19DCS106)

Prof. Rima Patel

Assistant Professor

Department of Computer Science and Engineering,

DEPSTAR/FTE, CHARUSAT-Changa.

ABSTRACT

This project promotes an approach for the Human Computer Interaction (HCI) where cursor movement can be controlled using a real-time camera, it is an alternative to the current methods including manual input of buttons or changing the positions of a physical computer mouse. Instead, it utilizes a camera and computer vision technology to control various mouse events and is capable of performing every task that the physical computer mouse can.

The Virtual Mouse color recognition program will constantly be acquiring real-time images where the images will be undergone a series of filtration and conversion. Whenever the process is complete, the program will apply the image processing technique to obtain the coordinates of the targeted colors position from the converted frames. After that, it will proceed to compare the existing colors within the frames with a list of color combinations, where different combinations consist of different mouse functions. If the current colors combination found a match, the program will execute the mouse function, which will be translated into an actual mouse function to the users' machine.

ACKNOWLEDGEMENT

We, the developers of “Virtual Pointer”, with immense pleasure and commitment would like to present the project assignment. The development of this project has given me wide opportunity to think, implement and interact with various aspects of management skills as well as the new emerging technologies.

Every work that one completes successfully stands on the constant encouragement, good will and support of the people around. We hereby avail this opportunity to express my gratitude to number of people who extended their valuable time, full support and cooperation in developing the project.

We express deep sense of gratitude towards our project guide Prof. Rima Patel for the support during the whole session of study and development. It is because of her, that we were prompted to do hard work, adopting new technologies.

We would also like to thank our friend Pratik Gupta for his guidelines throughout the development phase of the project. He helped me, whenever I was stuck in the Image Processing concepts.

They altogether provided me valuable inputs and guidance, and without them it would not have been possible to achieve my goal.

Thanks,
Smit Patel(19DCS103)
Yash Patel(19DCS106)

TABLE OF CONTENTS

1.	**INTRODUCTION**
2.	**PROJECT STATEMENT**
3.	**MOTIVATION**
4.	**PROJECT SCOPE**
5.	**PROJECT OBJECTIVE**
6.	**HARDWARE & SOFTWARE REQUIREMENTS**
7.	**FUNCTIONAL BLOCK DIAGRAM**
8.	**IMPLEMENTATION SCREENSHOTS**
9.	**CODE EXPLAINED**
10.	**LIMITATIONS**
11.	**FUTURE ENHACEMENTS**
12.	**CONCLUSION**
13.	**REFERENCES**

1. INTRODUCTION

A mouse, in computing terms is a pointing device that detects two-dimensional movements relative to a surface. This movement is converted into the movement of a pointer on a display that allows to control the Graphical User Interface (GUI) on a computer platform. There are a lot of different types of mouse that have already existed in the modern day's technology, there's the mechanical mouse that determines the movements by a hard rubber ball that rolls around as the mouse is moved. Years later, the optical mouse was introduced that replace the hard rubber ball to a LED sensor to detects table top movement and then sends off the information to the computer for processing. On the year 2004, the laser mouse was then introduced to improve the accuracy movement with the slightest hand movement, it overcome the limitations of the optical mouse which is the difficulties to track high-gloss surfaces. However, no matter how accurate can it be, there are still limitations exist within the mouse itself in both physical and technical terms. For example, a computer mouse is a consumable hardware device as it requires replacement in the long run, either the mouse buttons were degraded that causes inappropriate clicks, or the whole mouse was no longer detected by the computer itself. Despite the limitations, the computer technology still continues to grow, so does the importance of the human computer interactions. Ever since the introduction of a mobile device that can be interact with touch screen technology, the world is starting to demand the same technology to be applied on every technological device, this includes the desktop system. However, even though the touch screen technology for the desktop system is already exist, the price can be very steep. Therefore, a virtual human computer interaction device that replaces the physical mouse or keyboard by using a webcam or any other image capturing devices can be an alternative way for the touch screen. This device which is the webcam will be constantly utilized by a software that monitors the gestures given by the user in order to process it and translate to motion of a pointes, as similar to a physical mouse.

1.1. Review of The Physical Mouse

It is known that there are various types of physical computer mouse in the modern technology, the following will discuss about the types and differences about the physical mouse.

1.2. Mechanical Mouse

Known as the trackball mouse that is commonly used in the 1990s, the ball within the mouse are supported by two rotating rollers in order to detect the movement made by the ball itself. One roller detects the forward/backward motion while the other detects the left/right motion. The ball within the mouse are steel made that was covered with a layer of hard rubber, so that the detection are more precise. The common functions included are the left/right buttons and a scroll-wheel. However, due to the constant friction made between the mouse ball and the rollers itself, the mouse is prone to degradation, as overtime usage may cause the rollers to degrade, thus causing it to unable to detect the motion properly, rendering it useless. Furthermore, the switches in the mouse buttons are no different as well, as long-term usage may cause the mechanics within to be loosed and will no longer detect any mouse clicks till it was disassembled and repaired.

The following table describes the advantages and disadvantages of the Mechanical Mouse:

Advantage	Disadvantage
• Allows the users to control the computer system by moving the mouse. • Provides precise mouse tracking movements	• Prone to degradation of the mouse rollers and button switches, causing to be faulty. • Requires a flat surface to operate.

1.3 Optical and Laser Mouse

A mouse that commonly used in these days, the motions of optical mouse rely on the Light Emitting Diodes (LEDs) to detect movements relative to the underlying surface, while the laser mouse is an optical mouse that uses coherent laser lights. Comparing to its predecessor, which is the mechanical mouse, the optical mouse no longer rely on the rollers to determine its movement, instead it uses an imaging array of photodiodes. The purpose of implementing this is to eliminate the limitations of degradation that plagues the current predecessor, giving it more durability while offers better resolution and precision. However, there's still some downside, even-though the optical mouse is functional on most opaque diffuse surface, it's unable to detect motions on the polished surface. Furthermore, long term usage without a proper cleaning or maintenance may leads to dust particles trap between the LEDs, which will cause both optical and laser

mouse having surface detection difficulties. Other than that, it's still prone to degradation of the button switches, which again will cause the mouse to function improperly unless it was disassembled and repaired.

The following table describes the advantages and disadvantages of the Optical and Laser Mouse:

Advantages	Disadvantages
• Allows better precision with lesser hand movements. • Longer life-span.	• Prone to button switches degradation. • Does not function properly while on a polished surface.

2. PROJECT STATEMENT

It's no surprised that every technological device has its own limitations, especially when it comes to computer devices. After the review of various type of the physical mouse, the problems are identified and generalized. The following describes the general problem that the current physical mouse suffers:

- Physical mouse is subjected to mechanical wear and tear.
- Physical mouse requires special hardware and surface to operate.
- Physical mouse is not easily adaptable to different environments and its performance varies depending on the environment.
- Mouse has limited functions even in present operational environments.
- All wired mouse and wireless mouse have its own lifespan.

3. MOTIVATION

It is fair to say that the Virtual Mouse will soon to be substituting the traditional physical mouse in the near future, as people are aiming towards the lifestyle where that every technological device can be controlled and interacted remotely without using any peripheral devices such as the remote, keyboards, etc. it doesn't just provide convenience, but it's cost effective as well.

3.1. Convenient

It is known in order to interact with the computer system, users are required to use an actual physical mouse, which also requires a certain area of surface to operate, not to mention that it suffers from cable length limitations. Virtual Mouse requires none of it, as it only a webcam to allow image capturing of user's hand position in order to determine the position of the pointers that the user want it to be. For example, the user will be able to remotely control and interact the computer system by just facing the webcam or any other image capturing devices and moving your fingers, thus eliminating the need to manually move the physical mouse, while able to interact with the computer system from few feet away.

3.2. Cost Effective

A quality physical mouse is normally cost from the range of 30 ringgit to a hefty 400 ringgit, depending on their functionality and features. Since the Virtual Mouse requires only a webcam, a physical mouse are no longer required, thus eliminating the need to purchase one, as a single webcam is sufficient enough to allow users to interact with the computer system through it, while some other portable computer system such as the laptop, are already supplied with a built-in webcam, could simply utilize the Virtual Mouse software without having any concerns about purchasing any external peripheral devices.

4. PROJECT SCOPE

Virtual Mouse that will soon to be introduced to replace the physical computer mouse to promote convenience while still able to accurately interact and control the computer system. To do that, the software requires to be fast enough to capture and process every image, in order to successfully track the user's gesture. Therefore, this project will develop a software application with the aid of the latest software coding technique and the open-source computer vision library also known as the OpenCV.

The scope of the project is as below:

- Real time application.
- User friendly application.
- Removes the requirement of having a physical mouse.

The process of the application can be started when the user's gesture was captured in real time by the webcam, which the captured image will be processed for segmentation to identify which pixels values equals to the values of the defined color. After the segmentation is completed, the overall image will be converted to Binary Image where the identified pixels will show as white, while the rest are black. The position of the white segment in the image will be recorded and set as the position of the mouse pointer, thus resulting in simulating the mouse pointer without using a physical computer mouse. The software application is compatible with the Windows platform. The functionality of the software will be coded with Python programming language code with the integration of an external library that does the image processing known as the OpenCV.

5. PROJECT OBJECTIVE

The purpose of this project is to develop a Virtual Mouse application that targets a few aspects of significant development. For starters, this project aims to eliminate the needs of having a physical mouse while able to interact with the computer system through webcam by using various image processing techniques. Other than that, this project aims to develop a Virtual Mouse application that can be operational on all kind of surfaces and environment.

The following describes the overall objectives of this project:

I. To design to operate with the help of a webcam. The Virtual Mouse application will be operational with the help of a webcam, as the webcam are responsible to capture the images in real time. The application would not work if there are no webcam detected.

II. To design a virtual input that can operate on all surface. The Virtual Mouse application will be operational on all surface and indoor environment, as long the users are facing the webcam while doing the motion gesture.

III. To program the camera to continuously capturing the images, which the images will be analyzed, by using various image processing techniques. As stated above, the Virtual Mouse application will be continuously capturing the images in real time, where the images will be undergoing a series of process, this includes HSV conversion, Binary Image conversion, salt and pepper noise filtering, and more.

IV. To convert hand gesture/motion into mouse input that will be set to a particular screen position. The Virtual Mouse application will be programmed to detect the position of the defined colors where it will be set as the position of the mouse pointers. Furthermore, a combination of different colors may result in triggering different types of mouse events, such as the right/left clicks, scroll up/down, and more.

6. HARDWARE & SOFTWARE REQUIREMENTS

6.1. Hardware Requirement

The following describes the hardware needed in order to execute and develop the Virtual Mouse application:

- **Computer Desktop or Laptop**

The computer desktop or a laptop will be utilized to run the visual software in order to display what webcam had captured. A notebook which is a small, lightweight and inexpensive laptop computer is proposed to increase mobility.
System will be using

PROCESSOR	INTEL CORE i3 & Above
MAIN MEMORY	4GB RAM
HARD DISK	320GB
DISPLAY	14" Monitor

- **Webcam**

Webcam is utilized for image processing, the webcam will continuously taking image in order for the program to process the image and find pixel position.

6.2. Software Requirement

The following describes the software needed in-order to develop the Virtual Mouse application:

Python

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

Open CV Library

OpenCV are also included in the making of this program. OpenCV (Open Source Computer Vision) is a library of programming functions for real time computer vision. OpenCV have the utility that can read image pixels value, it also has the ability to create real time eye tracking and blink detection.

Software will be using:

OS	Window 7 Ultimate 64-bit
Language	Python
Tool Used	OpenCV

7. FUNCTIONAL BLOCK DIAGRAM

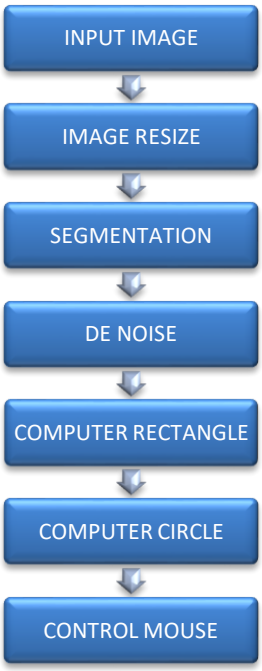

Figure 7.1

IMAGE RESIZE

- Map camera coordinates to screen coordinates.

SEGMENTATION

- Separate the Green objects from a complex background.
- We converted from BGR color space to HSV color space.

DENOISE

- Need to delete noisy pixels from the image.
- We use an image morphology algorithm that performs image erosion and image dilation to eliminate noise.
- Erosion trims down the image area where the hand is not present.
- Dilation expands the area of the Image pixels which are not eroded.

COMPUTE RECTANGLE

- We need to draw a rectangle around the objects and find the centre of the rectangle through which we will pass a line connecting the other rectangle in open gesture while a single rectangle will be drawn around the object in close gesture.

COMPUTE CIRCLE

- We need to find the centre of the line joining both rectangles and that centre will be a reference point at which mouse movements will occur and we draw a circle at that point to indicate that point.

CONTROL MOUSE

- In open gesture, we simply perform mouse movements and in close gesture we have performed left click operation of mouse.
- To control the speed of mouse movements and unnecessary shaking of cursor we have introduced a damping factor which will dampen the mouse movements and its value must be greater than one.

8. IMPLEMENTATION SCREENSHOTS:

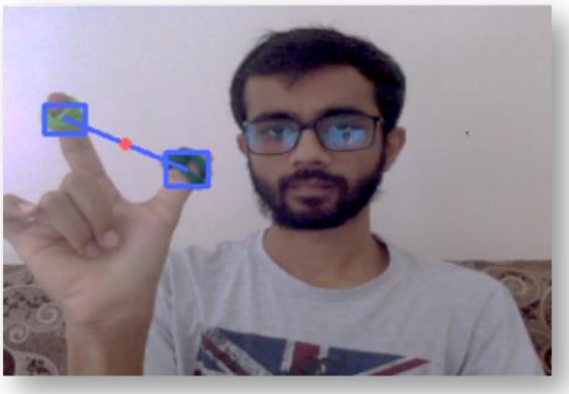

Figure 8.1

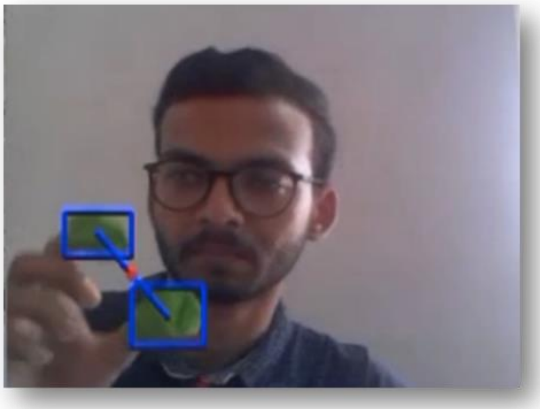

Figure 8.2

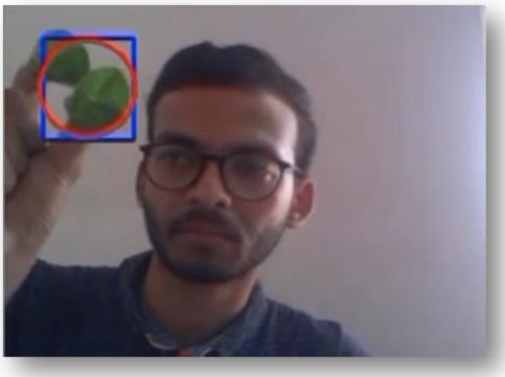

Figure 8.3

Figure 8.4

9. CODE EXPLAINED

- **Import libraries:**

```
import cv2
import numpy as np
from pynput.mouse import Button ,Controller
import wx
```

Figure 9.1

These are the libraries that we will be using. ***pynput*** to control mouse movements and clicking, ***wx*** to get the display resolution of the monitor, ***numpy*** to do mathematical operations / calculations and ***cv2*** to import OpenCV

- **Global variables Setup:**

Now, we need to setup global variables and objects. mouse object is for mouse movements and to get the screen resolution we need an ***wx*** app then we can use the ***wx.GetDisplaySize()*** to get the screen resolution. Lastly, we are setting some variables ***camx***, ***camy*** to set the captured image resolution. We will be using it later in image resize function.

```
mouse=Controller()
app=wx.App(False)
(sx,sy)=wx.GetDisplaySize() #Get width and height of laptop screen/monitor
cap=cv2.VideoCapture(0) #Captures video from primary camera
(camx,camy)=(320,240) #changing default set resolution of webcam t0 340 x 240
cap.set(3,camx) #objectName.set( ____,valuetochange) #3 represents width of image window
cap.set(4,camy) #objectName.set( ____,valuetochange) #4 represents height of image window
DampingFactor=2 #Damping factor must be greater than 1
pinchFlag=0 #We introduced pinchFlag to solve problem of continous left clicking in close gesture.
openx,openy,openw,openh=(0,0,0,0)
mLocOld=np.array([0,0]) #Setting initial values to zero
```

Figure 9.2

- **Color Detection**

Now, to detect color we need to know what is color in pixels of an image. Images are made of tiny dots of pixels each having a color and we can define those colors in terms of HSV -> Hue, Saturation, Value.

We need to write a program that we only want green color object to be detected rest of the colors are to be ignored. To do that, we need to decide a range for HSV value for Green (as there are lots of variation of green color). So, we declared these limits for the HSV values of each pixels. Now we will create a new binary image of same size as original image, we will call it mask and we'll make sure only those pixels that are in this HSV range will be allowed to be in the mask. That way only green objects will be in the mask.

```
#range for HSV (green color)
lowerBound=np.array([33,70,30])
upperBound=np.array([102,255,255])
```

Figure 9.3

First, we will read a frame from the camera then we resize it to make it a small fixed size for faster processing then finally we will convert this image to HSV format.

```
_,img=cap.read() #reads an images from webcam
img=cv2.resize(img,(340,240)) #resize of image for faster processing
imgHSV=cv2.cvtColor(img,cv2.COLOR_BGR2HSV) #Changes BGR to HSV for erasing noises in the image
mask=cv2.inRange(imgHSV,lowerBound,upperBound) #Creating a mask to erase noises in the image to detect object clearly
```

Figure 9.4

After, this we will be creating the filter which will create the mask for green color

- **Filtering the Mask**

Figure 9.5

We have some false positives in the mask. Those are the noises which are not good for object tracking. We have to clean to make out tracker work otherwise we will be seeing object marked in random places. To do that we need to do some morphological operation called opening and closing. Opening will remove all the dots randomly popping here and there and Closing will close the small holes that are present in the actual object. Then, we will store all the boundary points of the green objects that are detected with the use of findContours() method.

Figure 9.6

Figure 9.7

```
#using morphology to erase noise as maximum as possible
maskOpen=cv2.morphologyEx(mask,cv2.MORPH_OPEN, kernelOpen)
maskClose=cv2.morphologyEx(maskOpen,cv2.MORPH_CLOSE,kernelClose)
maskFinal=maskClose
_,conts,_=cv2.findContours(maskFinal,cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_NONE) #Stores all the boundary points of the green object detected
```

Figure 9.8

But, before doing that we need a 2d matrix called Kernel which is basically to control the effects of opening and closing

```
#Kerenel
kernelOpen=np.ones((5,5))
kernelClose=np.ones((20,20))
```

Figure 9.9

- **Implement the Open Gesture**

To Implement the open gesture, we need to draw a rectangle around the objects and find the centre of the rectangle through which we will pass a line connecting the other rectangle. Then, we need to find the centre of the line joining both rectangles and that centre will be a reference point at which mouse movements will occur and we draw a circle at that point to indicate that point.

Now we need to position our mouse curser according to the calculated coordinates.

In the above code, first we are doing a mouse release to ensure the mouse left button is not pressed. Then we are converting the detected coordinate from camera resolution to the actual screen resolution. After that we set the location as the ***mouse.position*** but to move the mouse it will take time for the curser so we have to wait till the curser reaches that point. So, we started a loop and we are not doing anything else until the current mouse location is same as assigned mouse location.

```
if(len(conts)==2):

    #if the button is pressed we need to release it first because in open gesture we need to only move the mouse.
    if(pinchFlag==1):
        pinchFlag=0
        mouse.release(Button.left)

    #drawing the rectagle around both green objects
    x1,y1,w1,h1=cv2.boundingRect(conts[0])
    x2,y2,w2,h2=cv2.boundingRect(conts[1])
    cv2.rectangle(img,(x1,y1),(x1+w1,y1+h1),(255,0,0),2)
    cv2.rectangle(img,(x2,y2),(x2+w2,y2+h2),(255,0,0),2)

    #the line between the center of the both rectangles
    [cx1,cy1]=[int(x1+w1/2),int(y1+h1/2)]
    [cx2,cy2]=[int(x2+w2/2),int(y2+h2/2)]
    cv2.line(img,(cx1,cy1),(cx2,cy2),(255,0,0),2)

    #the center of that line is the reference open at which we perform mouse movements
    [cx,cy]=[int((cx1+cx2)/2),int((cy1+cy2)/2)]
    cv2.circle(img,(cx,cy),2,(0,0,255),2) #Drawing a circle at that point to indicate that point

    mouseLoc=np.array([cx,cy])/DampingFactor #To increase smoothness of mouse movement and unnecessary movement we introduced damping factor
    [x,y] = mouseLoc
    mouse.position=(sx-int((x*sx)/camx),int((y*sy)/camy))
    while mouse.position!=(sx-int((x*sx)/camx),int((y*sy)/camy)):
        pass

    #these variables were added so that we get the outer rectangle that combines both objects
    openx,openy,openw,openh=cv2.boundingRect(np.array([[[x1,y1],[x1+w1,y1+h1],[x2,y2],[x2+w2,y2+h2]]]))
```

Figure 9.10

- ## Implement the Close Gesture

The above code is similar to the open gesture, but the difference is we only have one object here so we only need to calculate the centre of it. And that will be where we will position our mouse pointer. Also, we are performing a mouse press operation instead of mouse release operation. Rest of the part is same as of Open Gesture.

```
elif(len(conts)==1):
    x,y,w,h=cv2.boundingRect(conts[0])
    # we check first and we allow the press of left mouse button  if it's not pressed yet
    #we did that to avoid the continues pressing of left button of mouse
    if(pinchFlag==0):

        if(abs((w*h-openw*openh)*100/(w*h))<30): #if the difference between the combined rectangle for both object
                                                 # and the outer rectangle is not more than 30% then only we perform left click
            pinchFlag=1
            mouse.press(Button.left)
            openx,openy,openw,openh=(0,0,0,0)

    #this else was added so that if there's only one object detected it will not act as a mouse and we stop movement of mouse cursor
    else:
      #getting rectangle coordinates and drawing it
        x,y,w,h=cv2.boundingRect(conts[0])
        cv2.rectangle(img,(x,y),(x+w,y+h),(255,0,0),2)

        #getting the center of the circle that will be inside the outer rectangle
        [cx,cy]=[int(x+w/2),int(y+h/2)]
        cv2.circle(img,(cx,cy),int((w+h)/4),(0,0,255),2) #drawing that circle in is bounded inside the rectangle and radius of it is (width+height)/4

        mouseLoc=np.array([cx,cy])/DampingFactor  #To increase smoothness of mouse movement and unnecessary movement we introduced damping factor
        [x,y] = mouseLoc
        mouse.position=(sx-int((x*sx)/camx),int((y*sy)/camy))
        while mouse.position!=(sx-int((x*sx)/camx),int((y*sy)/camy)):
            pass
```

Figure 9.11

- **Some Fine Tuning:**

I. **ISSUE 1 RESOLVE:** We won't be able to drag anything because in close gesture we are continuously performing ***mouse.press*** operation which will result in continuous multiple clicks while dragging. To solve this problem, we will be putting a flag called "**pinchFlag**" and we will set that 1 once we perform a click operation and we won't perform mouse press operation anymore until the flag is 0 again.

II. **ISSUE 2 RESOLVE:** To avoid unnecessary movement of cursor we have introduced damping factor which will dampen mouse movements.

III. **ISSUE 3 RESOLVE:** If by chance one of the objects goes out of frame of the camera then in order to solve the problem of clicking we have introduced a third rectangle bounding both the rectangles of the object and if the area of both the combined rectangle and third rectangle are almost equal then only we perform mouse click operation.

IV. **ISSUE 4 RESOLVE:** In order to solve problem of click operation if only object is detected we added an else statement inside an else if statement. So as to perform clicking operation ONLY if two objects are detected.

10. LIMITATIONS

In this project, there are several existing problems that may hinder the results of color recognitions. One of the problems is the environmental factor during the recognition phase takes place.

The recognition process is highly sensitive on the intensity of brightness, as immense brightness or darkness may cause the targeted colors to be undetected within the captured frames. Besides that, distance is also the one of the problems that may affect the color recognition results, as the current detection region can support up to 25cm radius, any display of colors exceeds the mentioned distance will be considered as a noise and be filtered off. Furthermore, the performance of the program is highly dependent on the users' hardware, as processor speed and/or resolutions taken from the webcam could have an effect on performance load.

Therefore, the slower the processing speed and/or the higher the resolutions, the longer time are required to process a single frame.

11. FUTURE ENHACEMENTS

There are several features and improvements needed in order for the program to be more user friendly, accurate, and flexible in various environments. The following describes the improvements and the features required:

A. Introducing More Features

We can introduce more features like zoom in, zoom out, scroll up, scroll down, and left click operations.

B. Better Performance

The response time are heavily relying on the hardware of the machine, this includes the processing speed of the processor, the size of the available RAM, and the available features of webcam. Therefore, the program may have better performance when it's running on a decent machine with a webcam that performs better in different types of lightings.

12. CONCLUSION

In conclusion, it’s no surprised that the physical mouse will be replaced by a virtual non-physical mouse in the Human-Computer Interactions (HCI), where every mouse movements can be executed with a swift of your fingers everywhere and anytime without any environmental restrictions. This project had developed a color recognition program with the purpose of replacing the generic physical mouse without sacrificing the accuracy and efficiency, it is able to recognize color movements, combinations, and translate them into actual mouse functions.

Due to accuracy and efficiency plays an important role in making the program as useful as an actual physical mouse, a few techniques had to be implemented.
First and foremost, the coordinates of the colors that are in charge of handling the cursor movements are averaged based on a collection of coordinates, the purpose of this technique is to reduce and stabilize the sensitivity of cursor movements, as slight movement might lead to unwanted cursor movements. Other than that, several color combinations were implemented with the addition of distance calculations between two colors within the combination, as different distance triggers different mouse functions. The purpose of this implementation is to promote convenience in controlling the program without much of a hassle. Therefore, actual mouse functions can be triggered accurately with minimum trial and errors.

Furthermore, to promote efficient and flexible tracking of colors, calibrations phase was implemented, this allows the users to choose their choices of colors on different mouse functions, as long the selected colors doesn't fall within the same/similar RGB values (e.g. blue and sky-blue). Other than that, adaptive calibrations were also implemented as well, it is basically allows the program to save different set of HSV values from different angles where it will be used during the recognition phase.

In Overall, the modern technologies have come a long way in making the society life better in terms of productivity and lifestyle, not the other way around. Therefore, societies must not mingle on the past technologies while reluctant on accepting changes of the newer one. Instead, it’s advisable that they should embrace changes to have a more efficient, and productive lifestyle.

13. REFERENCES

- https://www.tutorialspoint.com/python/index.htm
- https://www.tutorialspoint.com/opencv/index.htm
- https://docs.opencv.org/3.4/dd/d49/tutorial_py_contour_features.html
- https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_gui/py_video_display/py_video_display.html
- https://pynput.readthedocs.io/en/latest/mouse.html
- https://www.tutorialspoint.com/wxpython/index.htm
- https://www.tutorialspoint.com/numpy/index.htm